MAHDI Abderaouf
SERRE Lohan

Master 1 Informatique GL

Rapport Robot de réconfort

HAI716I: Programmation orientée agents



Date du rendu : Octobre 2026
2026-2027

Sommaire

I. Architecture.....2
II. La stratégie de replanification.....2
III. Critique de la règle de repli.....3
IV. Trois échecs du robot..... 4
A. Le robot explore des impasses..... 4
B. Le robot repasse devant des casiers déjà observés.....4
C. Le robot ne reconnaît pas certains mots..... 4

I. Architecture

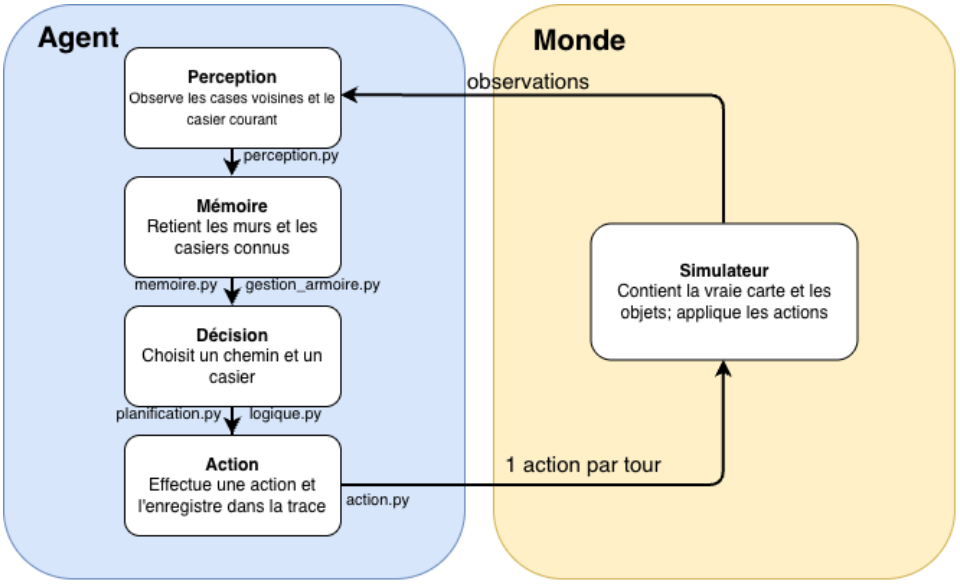


Figure 1. Architecture du robot et échanges entre l'agent et le simulateur

Notre programme est organisé autour de deux parties principales : l'agent et le simulateur.
Le simulateur (simulateur.py) représente l'environnement réel, il connaît la carte complète, les obstacles et le contenu des casiers. A l'inverse, l'agent (agent.py) ne peut pas accéder directement à ces informations. Il doit découvrir progressivement son environnement grâce aux perceptions et mémoriser ce qu'il apprend.

A chaque tour, le module perception.py récupère les informations accessibles au robot, notamment les cases voisines et le contenu du casier courant lorsqu'il se trouve près de l'armoire. Ces informations sont enregistrées dans memoire.py pour la carte et dans gestion_armoire.py pour les casiers. Le robot utilise ensuite planification.py pour calculer ses déplacements et logique.py pour reconnaître les émotions et déterminer les casiers à rechercher.

Enfin, action.py enregistre l'action dans la trace, puis la transmet au simulateur pour qu'elle soit exécutée. Une seule action est réalisée par tour, avant que le robot observe à nouveau son environnement. Les fichiers JSON sont également vérifiés au démarrage par validation.py

II. La stratégie de replanification

Le robot utilise l'algorithme BFS pour calculer ses déplacements. Comme il ne connaît pas les murs intérieurs au départ, il suppose que les cases encore inconnues sont libres. Il peut ainsi prévoir un chemin vers sa destination sans avoir exploré tout l'appartement.

Après chaque déplacement, le robot observe les cases voisines et met à jour sa carte mentale. Il recalcule ensuite entièrement son chemin avec les nouvelles informations, même si le trajet précédent reste possible. Il ne répare donc pas son ancien plan, il effectue une nouvelle recherche BFS.

Cette méthode est simple à mettre en œuvre et permet de réagir aux obstacles. Elle a cependant un coût : chaque recherche BFS parcourt au maximum un nombre de cases proportionnel à la taille de la grille, soit O(hauteur * largeur). De plus, comme le robot ne connaît pas tous les murs, il peut s'engager dans des impasses et effectuer des détours. Par exemple, sur notre carte acces_difficile, il effectue 100 déplacements pour atteindre le dictionnaire, alors qu'un trajet de 52 déplacements existe avec une connaissance complète de la carte.

III. Critique de la règle de repli

La règle actuelle permet au robot de chercher un objet dans les émotions voisines lorsque les casiers de l'émotion demandée sont vides. Cette méthode est simple et permet de toujours suivre le même ordre de recherche. Cependant, deux émotions proches sur la roue ne nécessitent pas forcément les mêmes objets.

Par exemple, une personne triste peut recevoir un désodorisant rangé dans la catégorie dégoût. Le robot respecte alors les règles du projet mais l'objet choisi n'est pas vraiment adapté pour réconforter cette personne. Cette règle n'est donc pas toujours pertinente.

Pour l'améliorer, nous proposons de définir une liste d'émotions de remplacement pour chaque émotion. Par exemple, pour la tristesse le robot pourrait chercher un objet associé à la confiance puis à la joie. Il privilégierait ainsi des objets plus adaptés aux besoins du résident.

Pour mettre en place cette amélioration, il faudrait ajouter un champ replis dans le fichier JSON de l'armoire. Le fichier validation.py vérifierait ce nouveau champ. Le fichier logique.py utiliserait ces informations pour modifier l'ordre de recherche des casiers. Il faudrait également adapter agent.py et la vérification des traces à cette nouvelle règle.

IV. Trois échecs du robot

A. Le robot explore des impasses

Lors de nos tests, le robot emprunte parfois des chemins qui se terminent par un mur. Dans un appartement contenant plusieurs impasses, il effectue 60 déplacements pour rejoindre l'armoire depuis le dictionnaire. Pourtant, le trajet le plus court ne nécessite que 40 déplacements lorsque tous les murs sont connus.

Ce comportement s'explique par le fait que le robot considère les cases inconnues comme libres. Il découvre donc certaines impasses en essayant de les traverser. Pour limiter ces détours, il pourrait privilégier les chemins déjà explorés lorsqu'ils ne sont pas beaucoup plus longs.

B. Le robot repasse devant des casiers déjà observés

Lors de la sixième demande de notre cinquième scénario de test, le robot retourne examiner des casiers de la catégorie surprise qu'il avait déjà observés vides lors d'une demande précédente. Cela entraîne des déplacements supplémentaires du sélecteur.

Ce comportement vient du fait que la mémoire de l'armoire est réinitialisée à chaque nouvelle demande. De plus, le robot doit respecter un ordre de priorité qui ne correspond pas toujours au trajet le plus court. Une amélioration serait de mieux organiser ses déplacements et de conserver certaines observations entre les demandes lorsque le contenu des casiers n'a pas changé.

C. Le robot ne reconnaît pas certains mots

Lorsqu'un résident dit “Je suis écœuré par ce repas, vraiment triste”, le robot ne reconnaît pas le mot “écœuré”. Pourtant, sa forme sans accent ecoeure est présente dans le dictionnaire. Il reconnaît donc le mot “triste” et risque de choisir un objet qui ne correspond pas à l'émotion principale du résident.

Ce problème vient de la fonction de normalisation. Elle supprime les accents mais ne transforme pas le caractère œ en oe. Pour améliorer la reconnaissance des émotions, il faudrait également prendre en compte ces caractères particuliers.